BACKEND STACK · PYTHON · WEEK 1

Python Foundations

A study reference for **weeekone.py**. Keep it open next to the code: every section follows the file's own order and tells you which lines it covers.

How to use this guide

- Find the line you're on in the code, then find the section whose **Lines** badge covers it in the table of contents.
- **Output** boxes show what Python actually prints. They were captured by running your file with Python 3.13.
- JS boxes compare the idea with JavaScript, since you already know it.
- In your file boxes point out a mistake, a surprise, or an outdated comment in the code. They are there to help, not to grade you.
- The last pages are a one-page cheat sheet and a list of every fix suggested in this guide.

Contents

	TOPIC	LINES	PAGE
01	Comments, `print()` and Indentation	1–35	3
02	String Methods	39–50	4
03	Concatenation and %-Formatting (old style)	54–91	5
04	The `str.format()` Method	95–154	6
05	f-Strings (Formatted String Literals)	156–162	7
06	Variables and Multiple Assignment	166–175	8
07	Numeric Types: int, float, complex	180–192	9
08	Arithmetic Operators	198–206	10
09	Lists: Indexing, Slicing, Mutability	211–241	11
10	List Methods	245–340	12
11	Tuples	343–444	13
12	Dictionaries	447–524	14
13	Dictionary Methods	526–617	15
14	Booleans and Truthiness	620–657	16
15	Logical (Boolean) Operators	661–680	17
16	Assignment Operators	681–703	18
17	Comparison Operators	704–759	19
18	Type Conversion (Casting) and Sets	761–819	20
19	User Input	821–833	21
20	Conditional Statements: if / elif / else	835–864	22
21	Nested Conditionals	867–898	23
22	One-line if and the Ternary Expression	900–925	24
23	Membership Operators	928–973	25
24	The `while` Loop (and `while…else`)	975–1027	26
25	The `for` Loop (and `for…else`)	1032–1065	27
26	`range()` and Looping Over a Dictionary	1067–1098	28
27	Nested Loops	1100–1151	29
28	Loop Control: break, continue, pass	1157–1195	30
29	Iterating Dictionaries with `items()`	1198–1242	31
30	Defining Functions and `return`	1244–1264	32
31	Parameters and Arguments	1267–1315	33
32	Variadic Positional Arguments: `*args`	1317–1345	34
33	Default Parameter Values	1348–1360	35
34	Variadic Keyword Arguments: `**kwargs`	1363–1393	36
35	Combining name, *args and **kwargs	1396–1422	37
36	Variable Scope and `global`	1426–1449	38
37	Recursion	1452–1481	39
38	Lambda (Anonymous) Functions	1483–1505	40
39	Practice: Three Ways to Call `range()`	1510–1523	41
★	Cheat Sheet	whole file	42
✓	Suggested Fixes in Your File	summary	43

01 Comments, `print()` and Indentation

Lines 1–35 Syntax basics

Comments

Anything after a `#` on a line is a **comment**. Python ignores it completely. Lines 1–27 of your file are all commented out, so none of that code runs; it's kept as notes. Python has no special block-comment syntax like `/* … */`; you put `#` on every line (most editors do this for you with `Ctrl`+`/`).

The `print()` function

`print()` is a **built-in function** that writes text to the screen (the "standard output") and then moves to a new line. You can give it several values separated by commas; it prints them with a single space between each one.

```
print("my name is", name)     # line 22: two values -> joined with a space
print()                       # line 12: no values -> prints an empty line
```

Two optional settings you'll meet later: `sep` (what goes between values, default `" "`) and `end` (what goes at the end, default a newline `"\n"`). Example: `print("a", "b", sep="-")` prints `a-b`.

Indentation defines blocks

Lines 4–12 show the most important rule in Python: **indentation is the syntax**. A line ending with a colon `:` (like `if True:`) opens a block, and every line that belongs to that block must be indented further. When the indentation goes back, the block is over. There are no curly braces.

```
if True:
    if True:
        print()     # belongs to the inner if
    print("x")      # belongs to the outer if
print("y")          # outside both
```

COMING FROM JAVASCRIPT

In JS, `{ }` mark the block and indentation is only for humans. In Python, the indentation *is* the block, so a wrong indent is a real error (`IndentationError`) or, worse, silently changes what your code does. There are also no semicolons at the end of lines.

Multi-line strings (triple quotes)

Lines 31–35 use **triple quotes** `'''…'''` (or `"""…"""`). A triple-quoted string can span several lines and keeps every line break and space exactly as typed. It can also contain normal quotes freely, which is why `"pyhton"` prints with its double quotes.

OUTPUT

```
 i love "pyhton"

i love you
```

COMING FROM JAVASCRIPT

This is like a JS template literal written with backticks, but without `${}` substitution. Python uses f-strings for substitution (section 05).

02 String Methods

Lines 39–50 Built-in methods of `str`

A **string** (`str`) is a sequence of characters. A **method** is a function that belongs to an object and is called with a dot: `st.upper()`. Strings are **immutable**: a method never changes `st` itself, it returns a *new* string. That's why line 47 onward still sees the original spaces each time.

Your test string has 4 spaces at the start and 5 at the end:

```
st = "    hello i love 3d imges and 2d dimintions     "
```

METHOD	WHAT IT DOES	RESULT IN YOUR FILE
`capitalize()`	Uppercases the first character and lowercases the rest. Here the first character is a space, so nothing visible changes.	`" hello i love 3d…"`
`upper()`	Every letter to uppercase. (`lower()` is the opposite.)	`" HELLO I LOVE 3D…"`
`title()`	Uppercases the first letter of every "word". A word starts after any non-letter, including digits, which is why `3d` becomes `3D`.	`" Hello I Love 3D Imges And 2D…"`
`count(sub, start, end)`	How many times `sub` appears. `start` and `end` are optional; `end` is **excluded**, so `-1` means "stop before the last character".	`5`
`find(sub)`	Index of the first match, or `-1` if not found.	`10`
`index(sub)`	Same as `find`, but raises a `ValueError` if not found.	`10`
`strip()`	Removes whitespace from both ends.	`"hello i love … dimintions"`
`lstrip()` / `rstrip()`	Removes whitespace from the left only / right only.	spaces kept on one side
`split()`	Cuts the string into a **list** of words. With no argument it splits on any run of whitespace and ignores the edges. `split(",")` splits on commas.	`['hello', 'i', 'love', '3d', …]`

Why is `find("i")` equal to `10`? Indexes start at 0. Positions 0–3 are spaces, `hello` is 4–8, a space at 9, so `i` is at 10.

COMING FROM JAVASCRIPT

`upper()` = `toUpperCase()`, `strip()` = `trim()`, `lstrip/rstrip` = `trimStart/trimEnd`, `find()` = `indexOf()`, `split()` = `split()`. JS has no direct `title()` or `capitalize()`.

CHAINING

Because each method returns a new string, you can chain them: `st.strip().title()`. Your file does exactly this on line 829.

03 Concatenation and %-Formatting (old style)

Lines 54–91 String concatenation · printf-style formatting

Concatenation with `+`

`"My Name is: " + name` joins two strings into one (line 62). The `+` only works when **both sides are strings**. Line 63 is commented out because `"…" + age` (string + int) raises a `TypeError`. To fix it you'd convert first: `"…" + str(age)`.

COMING FROM JAVASCRIPT

JS silently converts: `"Age: " + 36` gives `"Age: 36"`. Python refuses to guess and raises an error. This is called **strong typing**.

The `%` operator (printf-style formatting)

This is Python's oldest formatting style. You put **placeholders** in the string and give the values after a `%`. One value can be given directly; several values must be in a tuple `( )`, in order.

```
print("My Name is: %s and My Age is: %d" % (name, age))
```

OUTPUT

```
My Name is: Osama and My Age is: 36
```

PLACEHOLDER	MEANING	EXAMPLE
`%s`	string (actually works with anything, it converts with `str()`)	`"%s" % "Osama"` → `Osama`
`%d`	integer (decimal). A float is cut down to an int.	`"%d" % 36` → `36`
`%f`	float, 6 decimal places by default	`"%f" % 10` → `10.000000`
`%.2f`	float with exactly 2 decimals (rounded)	`"%.2f" % 10` → `10.00`
`%.5s`	string **truncated** to the first 5 characters	`"%.5s" % myLongString` → `Hello`

IN YOUR FILE

The comment on line 71 says `%d => Number`. More precisely it means **integer**; `%f` is the one for decimal numbers. That's why line 68 prints the rank as `10.000000`.

WHICH STYLE SHOULD I USE?

You'll see `%` in older code, so it's good to recognise it. For new code, prefer f-strings (section 05).

04 The `str.format()` Method

Lines 95–154 Format specification mini-language

`.format()` is the second formatting style. Each `{}` in the string is a **replacement field**; the arguments of `.format()` fill them in order.

```
print("My Name is: {} My Age: {}".format(name, age))
```

```
OUTPUT
My Name is: Osama My Age: 36
```

The format spec: what goes after the colon

Inside the braces you can write `{index:spec}`. The part after `:` is the **format specification**. The same spec language is used by f-strings, so learning it here pays twice.

FIELD	MEANING	YOUR LINE → OUTPUT
`{:s}`	string	109 → `Osama`
`{:d}`	integer. Raises `ValueError` if you pass a float.	124 → `10`
`{:f}`	float, 6 decimals	125 → `10.000000`
`{:.2f}`	float, 2 decimals	126 → `10.00`
`{:.5s}`	first 5 characters only	132 → `Hello`
`{:.13s}`	first 13 characters	133 → `Hello Peoples`
`{:_d}`	integer with `_` as thousands separator	140 → `500_162_350_198`
`{:,d}`	integer with `,` as thousands separator	141 → `500,162,350,198`
`{1}`	use argument number 1 (counting from 0)	147 → `Hello Two Three One`
`{2:.2f}`	argument 2, as a float with 2 decimals	154 → `30.00`

Rearranging items with indexes (lines 143–154)

With `a, b, c = "One", "Two", "Three"`, the arguments are numbered `0`, `1`, `2`. `"{1} {2} {0}"` means "second, third, first":

```
OUTPUT
Hello One Two Three
Hello Two Three One
Hello Three One Two
Hello 30.00 10.0000 20.00000
```

The last line (154) mixes both ideas: argument `2` with 2 decimals, argument `0` with 4, argument `1` with 5.

> **UNDERSCORES IN NUMBERS**
> You can also write big numbers with underscores in your code for readability: `myMoney = 500_162_350_198` is the same number.

05 f-Strings (Formatted String Literals)

Lines 156–162 · Python 3.6+ · the modern way

An **f-string** is a string with the letter `f` right before the opening quote. Inside it, anything in `{ }` is a real Python **expression** that gets evaluated and inserted. You can use variables, maths, method calls, and the same `:spec` from section 04.

```
print(f"My Name is : {myName} and My Age is : {myAge}")
print(f"{myAge + 1}")          # expressions work: 37
print(f"{3.14159:.2f}")        # format specs work: 3.14
```

IN YOUR FILE

Line 161 is missing the `f`, so Python treats the braces as plain text and prints them literally:

```
OUTPUT
My Name is : {myName} and My Age is : {myAge}
My Name is : Osama and My Age is : 36
```

Line 162 has the `f` and works. Line 161 might be intentional to show the difference; if so, add a comment saying so. If you ever see `{name}` in your output, the missing `f` is the first thing to check.

COMING FROM JAVASCRIPT

`f"Hello {name}"` is the same idea as `` `Hello ${name}` ``. No dollar sign in Python: a `$` inside an f-string is just a normal character, which is why lines 849 and 965 print a literal `$` in front of the price.

06 Variables and Multiple Assignment

Lines 166–175 Dynamic typing · naming · `str.join()`

A **variable** is a name that refers to a value. You create one simply by assigning: `name = "moayad"`. There's no `let`, `const` or `var`, and no type declaration. Python is **dynamically typed**: the *value* has a type, the name doesn't, and the same name can later point to a value of another type.

Multiple assignment

```
a, b, c = 1, 2, 3              # a=1, b=2, c=3
name1, name2 = "ali", "omar"
```

The right side is really a tuple, and Python **unpacks** it into the names on the left. The counts must match, or you get a `ValueError`. A classic trick this enables: swapping two variables with `a, b = b, a`.

Joining a list into a string

```
arr = ["mari", "sara"]
print(",".join(arr))
```

```
OUTPUT
mari,sara
```

`join` is a **string** method: the separator is the object, the list is the argument. It's the reverse of `split()`. Every item must already be a string.

> **COMING FROM JAVASCRIPT**
> JS writes `arr.join(",")`; Python writes `",".join(arr)`. Your comment on line 174 is right: a Python **list** is the equivalent of a JS **array**.

> **NAMING STYLE**
> Python's style guide (PEP 8) uses `snake_case` for variables and functions: `my_name`, not `myName`. Your file mixes both, which works fine, but most Python code you read will use snake_case. Names are case-sensitive: `name` and `Name` are different variables.

07 Numeric Types: int, float, complex

Lines 180–192 `type()` · numeric conversion

TYPE	WHAT IT IS	EXAMPLE
`int`	whole numbers, any size (no overflow)	`5`, `-10`, `500162350198`
`float`	numbers with a decimal point	`5.6`, `100.5`
`complex`	real + imaginary part, written with `j`	`5+6j`

`type(x)` tells you the type of any value. It prints like `<class 'int'>` because in Python every type is a **class**.

```
print(int(b))       # int(5.6)  -> 5      cuts off the decimals (no rounding!)
print(float(a))     # float(5)  -> 5.0
print(complex(a))   # complex(5)-> (5+0j)
print(c.real)       # 5.0       real part of 5+6j
print(c.imag)       # 6.0       imaginary part
```

WATCH OUT

`int(5.6)` gives `5`, not `6`. `int()` **truncates** toward zero. Use `round(5.6)` if you want `6`.

COMING FROM JAVASCRIPT

JS has a single `number` type for everything. Python keeps integers and floats separate, and `5 == 5.0` is still `True`. Complex numbers are rare in backend work; just know they exist.

08 Arithmetic Operators

Lines 198–206 Listed only as comments in your file, so here they are with examples

OPERATOR	NAME	EXAMPLE	RESULT
`+`	addition	`7 + 2`	`9`
`-`	subtraction	`7 - 2`	`5`
`*`	multiplication	`7 * 2`	`14`
`/`	true division (always returns a float)	`8 / 2`	`4.0`
`//`	floor division (rounds **down**)	`7 // 2` `-7 // 2`	`3` `-4`
`%`	modulo (remainder)	`7 % 2`	`1`
`**`	exponent (power)	`2 ** 10`	`1024`

Order of operations follows maths: `**` first, then `* / // %`, then `+ -`. Use parentheses when in doubt: `(2 + 3) * 4` is `20`.

The `%` operator is used for real on line 1049: `number % 2 == 0` means "divides by 2 with nothing left over", which is how you test for an even number.

> **COMING FROM JAVASCRIPT**
> JS has no `//`; the equivalent is `Math.floor(7 / 2)`. In JS `8 / 2` is just `4`; in Python it's the float `4.0`. Python has no `++` or `--`; you write `x += 1`.

09 Lists: Indexing, Slicing, Mutability

Lines 211–241 Sequence type `list`

A **list** is an ordered, changeable collection written with square brackets. The five rules in your comments (lines 214–218) are exactly right: ordered, indexed from 0, mutable, duplicates allowed, mixed types allowed.

```
myAwesomeList = ["One", "Two", "One", 1, 100.5, True]
#   index:          0      1      2   3    4     5
#   negative:      -6     -5     -4  -3   -2    -1
```

Indexing

`list[i]` gets one item. **Negative indexes** count from the end: `[-1]` is the last item, `[-3]` is the third from the end. So line 225 prints `True` and line 226 prints `1`.

Slicing: `list[start:stop:step]`

A **slice** returns a new list. `start` is included, `stop` is **excluded**, and any part can be left out.

LINE	SLICE	MEANING	OUTPUT
228	`[1:4]`	index 1, 2, 3	`['Two', 'One', 1]`
229	`[:4]`	from the start up to 3	`['One', 'Two', 'One', 1]`
230	`[1:]`	from 1 to the end	`['Two', 'One', 1, 100.5, True]`
232	`[::1]`	everything, step 1 (a copy)	whole list
233	`[::2]`	every second item	`['One', 'One', 100.5]`
–	`[::-1]`	bonus: the list reversed	`[True, 100.5, 1, …]`

Changing items, including with slice assignment

Because lists are mutable you can assign to an index (`myAwesomeList[1] = 2`) or even to a slice. A slice assignment **replaces that whole range** with whatever list you give, even if it's a different length:

```
myAwesomeList[0:3] = ["A","B","C"]   # 3 items replaced by 3
myAwesomeList[0:3] = ["A"]           # 3 items replaced by 1: the list shrinks!
```

```
OUTPUT
['A', 'B', 'C', 1, 100.5, True]
['A', 1, 100.5, True]
```

COMING FROM JAVASCRIPT

`arr[-1]` does *not* work in JS (you'd use `arr.at(-1)`). `list[1:4]` is like `arr.slice(1, 4)` . Slice assignment is roughly `arr.splice(0, 3, "A")` .

10 List Methods

Lines 245–340 Methods of `list`

Unlike string methods, most list methods **change the list in place** and return `None`. So write `y.sort()` then `print(y)`, never `y = y.sort()` (that would set `y` to `None`).

METHOD	WHAT IT DOES	JS EQUIVALENT
`append(x)`	Adds **one** item to the end. If `x` is a list, the whole list becomes a single item (a nested list).	`push(x)`
`extend(iter)`	Adds **each item** of another list to the end.	`push(...arr)`
`remove(x)`	Removes the **first** item equal to `x`. `ValueError` if missing.	–
`sort()`	Sorts ascending. `sort(reverse=True)` sorts descending.	`sort()`
`reverse()`	Reverses the current order (does not sort).	`reverse()`
`clear()`	Empties the list.	`arr.length = 0`
`copy()`	Returns a new, independent (shallow) copy.	`[...arr]`
`count(x)`	How many times `x` appears.	–
`index(x)`	Index of the first `x`.	`indexOf(x)`
`insert(i, x)`	Inserts `x` **before** position `i`.	`splice(i, 0, x)`
`pop(i)`	Removes and **returns** the item at `i` (last item if no `i`).	`pop()` / `splice(i,1)`

Walking through your examples

append (254–264). After five appends, `myFriends` has 8 items, and the last one is the whole `myOldFriends` list. That's why you need two indexes to reach "Ali": `myFriends[7][2]`. Note `150.200` prints as `150.2`; trailing zeros aren't stored in a float.

```
OUTPUT
['Osama', 'Ahmed', 'Sayed', 'Alaa', 100, 150.2, True, ['Haytham', 'Samah', 'Ali']]
```

extend (268–275) flattens instead: `[1, 2, 3, 4, 'A', 'B', 'C', 'One', 'Two']`.

remove (279–281) removes only the first `"Osama"`; the other two stay.

sort (285–288) gives `[120, 100, 29, 17, 2, 1, -10]`. The commented line 286 shows sorting strings works too (alphabetically). What you can't do is sort a **mix** of numbers and strings: Python raises `TypeError` because it doesn't know whether `"A"` is bigger than `5`. `reverse()` on line 293 works on the mixed list because it doesn't compare anything.

copy (308–317) is the key lesson: after `c = b.copy()`, appending to `b` does not affect `c`. If you had written `c = b` instead, both names would point to the **same** list and both would show the 5.

insert (331–335). `insert(0, "Test")` puts it at the very front. `insert(-1, "Test")` puts it **before the last item**, not at the end:

```
OUTPUT
['Test', 1, 2, 3, 4, 5, 'A', 'Test', 'B']
```

pop (339–340). `g.pop(-3)` removes the third-from-last item and returns it, so `5` is printed.

11 Tuples

Lines 343–444 Immutable sequence type `tuple`

A **tuple** is like a list that can't be changed after it's created (**immutable**). Use it for fixed groups of values: coordinates `(x, y)`, an RGB colour, a database row, or several values returned from one function.

Syntax

```
t1 = ("Osama", "Ahmed")    # with parentheses
t2 = "Osama", "Ahmed"      # without: the COMMA makes the tuple
one = ("Osama",)           # one item needs a trailing comma
not_a_tuple = ("Osama")    # this is just a string in brackets!
```

Lines 391–401 test exactly this: both one-item tuples have `len` 1 and type `tuple`. Without the comma you'd get a string of length 5.

What works and what doesn't

- **Indexing and slicing** work like lists: `t[0]`, `t[-1]`, `t[1:3]`.
- **Assigning** to an item fails: `t[2] = "Three"` → `TypeError: 'tuple' object does not support item assignment` (lines 376–377).
- **Concatenation** with `+` builds a *new* tuple: `a + ("A", "B", True) + b` → `(1, 2, 3, 4, 'A', 'B', True, 5, 6)`.
- **Repetition** with `*` works on strings, lists and tuples (lines 416–422): `"Osama" * 6`, `[1, 2] * 6`, `("A", "B") * 6`. You use the same trick for separators: `print("=" * 50)`.
- Tuples have only two methods: `count(x)` and `index(x)`. `a.count(8)` → `2`, `b.index(7)` → `2`.

> **LINE 432**
> is commented because `"…" + b.index(7)` is string + int → `TypeError` (same rule as section 03). Lines 433 and 434 show the two correct ways: `.format()` and an f-string.

Tuple unpacking (destructuring)

```
a = ("A", "B", 4, "C")
x, y, _, z = a        # x="A", y="B", _=4 (ignored), z="C"
```

The `_` is a normal variable name, but by convention it means "I don't care about this value". The number of names must match the number of items. You can also collect the rest with a star: `first, *rest = a`.

> **COMING FROM JAVASCRIPT**
> JS has no tuple type. Unpacking is like array destructuring `const [x, y, , z] = a`, where JS uses an empty slot and Python uses `_`.

12 Dictionaries

Lines 447–524 Mapping type `dict` · key–value pairs

A **dictionary** stores **key: value** pairs in curly braces. You look things up by key instead of by position. It's the most-used data structure in backend work (JSON from an API becomes a dict in Python).

```
user = {
  "name": "Osama",
  "age": 36,
  "skills": ["Html", "Css", "JS"],
}
user["name"]          # 'Osama'   KeyError if the key doesn't exist
user.get("name")      # 'Osama'   returns None if the key doesn't exist
user.get("x", "N/A")  # 'N/A'     .get with a default value
```

Keys must be unique and **immutable** (strings, numbers, tuples). A list can't be a key. **Values** can be anything, including lists and other dicts. If you repeat a key, the last value wins.

Viewing keys and values

```
OUTPUT
dict_keys(['name', 'age', 'country', 'skills', 'rating'])
dict_values(['Osama', 36, 'Egypt', ['Html', 'Css', 'JS'], 10.5])
```

Nested ("two-dimensional") dictionaries

A value can itself be a dict. Chain the brackets to go deeper: `languages['Three']['name']` → `'Js'`. `len()` counts the top-level keys: `len(languages)` is 3, and `len(languages["Two"])` is 2. Lines 503–524 build the same shape by putting existing dict variables inside a new dict.

IN YOUR FILE

Line 455 says "Dict Is Not Ordered". That was true in very old Python, but since **Python 3.7 dictionaries keep insertion order**: keys come out in the order you added them. You still access items by key, not by position (`user[0]` doesn't work).

COMING FROM JAVASCRIPT

A dict looks like a JS object literal, but keys must be quoted (`"name": …`), and you can't use dot access: it's `user["name"]`, never `user.name`. The closest JS type is `Map`.

13 Dictionary Methods

Lines 526–617 Methods of `dict`

METHOD	WHAT IT DOES	YOUR OUTPUT
`clear()`	Removes all pairs.	`{}`
`d[key] = v`	Adds a new pair, or replaces the value if the key exists.	`{'name': 'Osama', 'age': 36}`
`update({...})`	Adds/overwrites several pairs at once from another dict.	`{…, 'country': 'Egypt'}`
`copy()`	Independent shallow copy. Changing `main` later doesn't touch `b`.	`b` stays `{'name': 'Osama'}`
`keys()` / `values()`	Views of all keys / all values.	`dict_keys([...])`
`setdefault(k, v)`	If `k` exists, returns its value. If not, **adds** `k: v` and returns `v`.	`36`, then `age` is in the dict
`popitem()`	Removes and returns the **last inserted** pair as a tuple.	`('age', 36)`
`items()`	View of all pairs as `(key, value)` tuples.	`dict_items([('name', 'Osama'), …])`
`dict.fromkeys(keys, v)`	Builds a new dict where every key gets the same value `v`.	`{'MyKeyOne': 'X', 'MyKeyTwo': 'X', …}`

"Views" are live

Lines 599–608 show a subtle point. `allItems = view.items()` is taken **before** `"age"` is added, yet printing it afterwards shows the age:

```
OUTPUT
dict_items([('name', 'Osama'), ('skill', 'XBox'), ('age', 36)])
```

That's because `keys()`, `values()` and `items()` return **views**: windows onto the dict, not snapshots. Wrap them in `list(...)` if you need a frozen copy.

> **ALSO USEFUL**
> `pop(key)` removes a specific key and returns its value. `del user["age"]` deletes a key. `"age" in user` checks whether a key exists (section 23).

14 Booleans and Truthiness

Lines 620–657 Type `bool` · truthy and falsy values

A **boolean** is one of two values: `True` or `False`, always with a capital letter. Comparisons produce booleans: `100 > 90` is `True`. Many methods do too, like `" ".isspace()` → `True` (line 628; it checks whether the string is only whitespace).

Truthiness: `bool(x)`

Every value can be treated as true or false, for example in an `if`. `bool()` shows you which:

FALSY (BECOME `FALSE`)	TRUTHY (BECOME `TRUE`)
`0`, `0.0`	Any non-zero number: `100`, `100.95`, `-1`
`""` and `''` (empty string)	Any non-empty string: `"Osama"`, even `" "`
`[]` empty list, `()` empty tuple, `{}` empty dict	Any non-empty collection: `[1, 2, 3]`
`False`	`True`
`None`	

A simple rule: **empty or zero is false, everything else is true**. This lets you write `if my_list:` instead of `if len(my_list) > 0:`.

`None` is Python's "no value", the result of a function that doesn't return anything.

> **COMING FROM JAVASCRIPT**
> Big difference: in JS, `[]` and `{}` are **truthy**. In Python, empty collections are **falsy**. Python has one `None` where JS has both `null` and `undefined`, and `NaN` is truthy in Python.

15 Logical (Boolean) Operators

Lines 661–680 `and` · `or` · `not`

OPERATOR	TRUE WHEN...	JS
`and`	**all** conditions are true	`&&`
`or`	**at least one** condition is true	`||`
`not`	flips the result	`!`

```
age, country, rank = 36, "Egypt", 10
age > 16 and country == "Egypt" and rank > 0   # True  (all three true)
age > 16 and country == "KSA"   and rank > 0   # False (one is false)
age > 40 or country == "Egypt"  or rank > 20   # True  (one is true)
not age > 16                                   # False
```

Short-circuiting: Python stops as soon as the answer is known. In `A and B`, if `A` is false, `B` is never checked. In `A or B`, if `A` is true, `B` is never checked.

Precedence: `not` runs first, then `and`, then `or`. Comparisons like `>` run before all three, which is why `not age > 16` means `not (age > 16)`.

16 Assignment Operators

Lines 681–703 Augmented assignment

An **augmented assignment** is a shortcut for "update a variable using its own value". Your comment on lines 697–698 describes it perfectly: `x = x - y` can be written `x -= y`.

SHORT FORM	LONG FORM	WITH X = 10, Y = 20
`x += y`	`x = x + y`	30
`x -= y`	`x = x - y`	**-10** (what line 701 prints)
`x *= y`	`x = x * y`	200
`x /= y`	`x = x / y`	0.5
`x **= 2`	`x = x ** 2`	100
`x %= 3`	`x = x % 3`	1
`x //= 3`	`x = x // 3`	3

> **COMING FROM JAVASCRIPT**
> Same operators as JS, except Python has no `x++` / `x--`. Use `x += 1`, as your while loops do.

17 Comparison Operators

Lines 704–759 Relational operators

OPERATOR	MEANING	`100 ? 100.00`	`100 ? 200`
`==`	equal	True	False
`!=`	not equal	False	True
`>`	greater than	False	False
`<`	less than	False	True
`>=`	greater than or equal	True	False
`<=`	less than or equal	True	True

The point your file tests with `100.00`: Python compares **values**, so the int `100` equals the float `100.00`. Strings compare too: `"apple" < "banana"` is `True` (alphabetical, and uppercase letters come before lowercase).

Python also lets you chain comparisons: `0 < age < 120` means `0 < age and age < 120`.

> **COMING FROM JAVASCRIPT**
> There's no `===` in Python, and you don't need it: `==` never converts types the weird JS way. `1 == "1"` is `False` in Python. The keyword `is` checks whether two names point to the same object; use it only for `None`: `if x is None:`.

18 Type Conversion (Casting) and Sets

Lines 761–819 `str()` · `tuple()` · `list()` · `set()` · `dict()`

Type conversion means creating a value of one type from a value of another. Each type's name is also a function that does the conversion. It always returns a new value; the original is unchanged.

FROM	`TUPLE(X)` / `LIST(X)` / `SET(X)` GIVES...
string `"Osama"`	one item per **character**: `('O', 's', 'a', 'm', 'a')`
list or tuple	the same items in the new container
set `{"A", "B", "C"}`	the items, in no guaranteed order
dict `{"A": 1, "B": 2}`	only the **keys**: `('A', 'B')`

`str(10)` turns a number into the text `"10"` (lines 767–769: the type goes from `int` to `str`).

`dict()` needs **pairs**: a tuple or list whose items each have exactly two parts (lines 815–819).

```
dict((("A", 1), ("B", 2), ("C", 3)))   # {'A': 1, 'B': 2, 'C': 3}
```

A new type appears: the set

Line 777 quietly introduces the **set**: `{"A", "B", "C"}`. Curly braces, but no colons. A set is an **unordered collection of unique items**: duplicates are removed automatically and there are no indexes.

```
set("Osama")      # {'s', 'a', 'O', 'm'}   only one 'a' survives
```

WATCH OUT

Because sets have no order, printing a set (or converting one to a list) can show the items in a different order each run. Your output showed `['C', 'A', 'B']`; yours may differ. And `{}` on its own is an empty **dict**, not an empty set. Use `set()` for that.

COMING FROM JAVASCRIPT

`str(x)` ≈ `String(x)`, `int("5")` ≈ `parseInt("5")`, `list("abc")` ≈ `Array.from("abc")`, and a Python set ≈ `new Set([...])`.

19 User Input

Lines 821–833 `input()` · cleaning text

`input(prompt)` shows the prompt, waits for the user to type something and press Enter, and returns what they typed **as a string, always**. Even if they type `36`, you get `"36"`. To use it as a number, convert it: `age = int(input("Age? "))`.

```
fName = input('What\'s Is Your First Name?')
fName = fName.strip().capitalize()            # "  moayad " -> "Moayad"
print(f"Hello {fName} {mName:.1s} {lName} Happy To See You.")
```

- `\'` is an **escape sequence**: a backslash lets you put a single quote inside a single-quoted string. Writing the string in double quotes avoids the need: `"What's your name?"`.
- `.strip().capitalize()` cleans whatever the user typed: removes stray spaces, then fixes the capitals.
- `{mName:.1s}` uses the truncation spec from section 04 to keep only the **first letter** of the middle name, so you get an initial.

Typing `moayad`, `ahmad`, `attalah` produces:

```
OUTPUT
Hello Moayad A Attalah Happy To See You.
```

SMALL POLISH

Add a space at the end of each prompt (`"First name? "`) so the user's typing doesn't stick to the question mark.

COMING FROM JAVASCRIPT

This is like the browser's `prompt()`, or `readline` in Node, but simpler: it just blocks until the user presses Enter.

20 Conditional Statements: if / elif / else

Lines 835–864 Control flow · making decisions

A **conditional statement** runs a block only when a condition is true. Python checks each condition from top to bottom and runs the **first** block whose condition is true, then skips the rest. `else` runs only if nothing above matched.

```
if uCountry == "Egypt":
  ...                       # 80 off
elif uCountry == "KSA":     # "elif" = "else if"
  ...                       # 60 off
elif uCountry == "Kuwait":
  ...                       # 50 off   <- this one runs
else:
  ...                       # 30 off
```

With `uCountry = "Kuwait"` and `cPrice = 100` the output is:

```
OUTPUT
Hello Osama Because You Are From Kuwait
The Course "Python Course" Price Is: $50
```

- Every `if`, `elif` and `else` line ends with a colon, and its block is indented.
- `\"` inside the f-string is an escaped double quote so it can appear inside a double-quoted string.
- `{cPrice - 50}` shows that f-strings can do maths.
- No parentheses are needed around the condition.

INDENTATION STYLE

This part of your file indents with 2 spaces; the top and bottom use 4. Both run, as long as each block is consistent with itself. The Python standard (PEP 8) is **4 spaces**, so it's a good habit to stick to that.

COMING FROM JAVASCRIPT

`if (x) { } else if (y) { } else { }` becomes `if x: … elif y: … else: …`. Python had no `switch` for a long time; since 3.10 it has `match` / `case`, which you'll meet later.

21 Nested Conditionals

Lines 867–898 An if inside an if

A **nested if** is a conditional inside another one's block. The inner check only happens if the outer one passed. Indentation shows the nesting level.

```
if uCountry == "Egypt" or uCountry == "KSA" or uCountry == "Qatar":
  if isStudent == "Yes":              # only checked for these 3 countries
    ...  # 90 off   <- runs (Egypt and a student)
  else:
    ...  # 80 off
elif uCountry == "Kuwait" or uCountry == "Bahrain":
  ...
else:
  ...
```

OUTPUT

```
Hi Osama Because U R From Egypt And Student
The Course "Python Course" Price Is: $10
```

CLEANER CONDITION

Instead of repeating `uCountry == …` three times, use membership (section 23): `if uCountry in ["Egypt", "KSA", "Qatar"]:`. A common mistake to avoid: `if uCountry == "Egypt" or "KSA":` is **always true**, because `"KSA"` on its own is a non-empty (truthy) string.

Storing yes/no as the string `"Yes"` works, but a real boolean is more natural: `isStudent = True`, then `if isStudent:`.

22 One-line if and the Ternary Expression

Lines 900–925 Conditional expression

One-line if statements (lines 904–908)

If a block has only one statement, you can write it on the same line after the colon:

```
if country == "Egypt" : print(f"The Weather in {country} Is 15")
elif country == "KSA" : print(f"The Weather in {country} Is 30")
else : print("Country is Not in The List")
```

This is still a normal `if` statement, just written compactly. With `country = "A"` it prints `Country is Not in The List`.

The real ternary: a conditional expression (line 923)

The **conditional expression** picks one of two *values* and can be used anywhere a value fits: inside `print()`, on the right of `=`, in an f-string. The order reads like English:

```
value_if_true  if  condition  else  value_if_false

print("Movie S Not Good 4U" if age < movieRate else "Movie S Good 4U And Happy Watching")
label = "adult" if age >= 18 else "minor"
```

Your comment on line 925 lays it out correctly. With `age = 18` and `movieRate = 18`, `18 < 18` is false, so the second value is printed.

IN YOUR FILE

The heading on line 901 says "Ternary Conditional Operator", but lines 906–908 are one-line `if` statements, not the ternary. The actual ternary is line 923. It helps to keep the two apart in your mind: a statement *does* something, an expression *produces a value*.

COMING FROM JAVASCRIPT

JS writes `cond ? a : b`. Python writes `a if cond else b`, with the condition in the middle.

23 Membership Operators

Lines 928–973 `in` · `not in`

`x in container` is `True` if `x` is found inside. `not in` is the opposite. It works on strings (substring), lists, tuples, sets and dicts (checks the **keys**).

```
"s" in "Osama"              # True
"A" in "Osama"              # False   case-sensitive: only lowercase 'a'
"Sayed" in friends          # True
"Mahmoud" not in friends    # False   Mahmoud IS in the list
"age" in user               # True if user has the KEY "age"
```

Lines 955–973 combine it with `if/elif` to find which discount group a country is in. `"Italy"` is in `countriesTwo`, so the output is `Hello You Have A Discount Equal To $50`. Notice how much shorter this is than chaining `==` with `or`.

> **COMING FROM JAVASCRIPT**
> `x in list` ≈ `arr.includes(x)`; `"s" in "Osama"` ≈ `"Osama".includes("s")`. Careful: JS's own `in` checks object keys (and array indexes), not values.

24 The `while` Loop (and `while…else`)

Lines 975–1027 Condition-controlled loop

A **while loop** repeats its block **as long as** the condition is true. It checks the condition before every round. Something inside the loop must eventually make the condition false, or you get an **infinite loop** (stop it with `Ctrl` + `C`).

```
a = 0
while a < 15:
  print(a)      # 0, 1, 2 ... 14
  a += 1        # without this line the loop never ends
print("Loop is Done")
```

`while False:` (line 992) never runs at all, because the condition is false from the start.

Looping over a list by index (lines 1002–1016)

```
while a < len(myF):                      # len(myF) is 10
  print(f"#{str(a + 1).zfill(3)} {myF[a]}")
  a += 1
else:
  print("All Friends Printed To Screen.")
```

- `len(myF)` returns how many items the list has.
- `str(a + 1).zfill(3)` turns the number into text and pads it with zeros on the left to 3 characters: `1` → `"001"`. (`zfill` is a string method, so the `str()` conversion is needed first.)

```
OUTPUT
#001 Os
#002 Ah
...
#010 Wa
All Friends Printed To Screen.
```

The `else` on a loop

Python loops can have an `else` block. It runs once when the loop finishes **normally** (the condition became false), and it is **skipped** if the loop was stopped with `break` (section 28). It's useful for "searched everything and didn't find it" logic.

> **COMING FROM JAVASCRIPT**
> Same as JS `while (a < 15) { … }`. JS has no loop-`else`. Python has no `do…while`.

25 The `for` Loop (and `for…else`)

Lines 1032–1065 Iterating over an iterable

Python's **for loop** goes through the items of an **iterable** (anything you can loop over: list, tuple, set, dict, string, range…) one by one. The loop variable holds the current item. You don't manage an index or a counter.

```
for number in myNumbers:
  if number % 2 == 0:
    print(f"The Number {number} Is Even.")
  else:
    print(f"The Number {number} Is Odd.")
else:
  print("The Loop Is Finished")      # runs once, after the last item
```

Looping over a **string** gives you one character at a time (lines 1061–1065):

```
OUTPUT
 [ O ]
 [ S ]
 [ A ]
 [ M ]
 [ A ]
```

The loop variable name is your choice (`number` , `letter` , `i` …). Pick one that describes a single item. The loop's `else` works exactly like on a `while` loop.

> **COMING FROM JAVASCRIPT**
> Python's `for x in items` is JS's `for (const x of items)` . It is **not** JS's `for…in` . There is no C-style `for (let i = 0; i < n; i++)` ; you use `range()` for that (section 26).

26 `range()` and Looping Over a Dictionary

Lines 1067–1098 Number sequences · dict iteration

`range(start, stop, step)`

`range()` produces a sequence of integers. Like slicing, `stop` is **excluded**. It doesn't build a list in memory; it generates the numbers as the loop asks for them.

CALL	NUMBERS
`range(5)`	0, 1, 2, 3, 4
`range(1, 101)`	1 … 100 (line 1074)
`range(0, 10, 2)`	0, 2, 4, 6, 8
`range(10, 0, -1)`	10, 9 … 1 (counting down)

Looping over a dictionary

A `for` loop over a dict gives you its **keys**. Use each key to fetch its value:

```
for skill in mySkills:
  print(f"My Progress in Lang {skill} Is: {mySkills.get(skill)}")
```

```
OUTPUT
My Progress in Lang Html Is: 90%
My Progress in Lang Css Is: 60%
...
```

Section 29 shows the cleaner way to get key and value together with `items()`.

27 Nested Loops

Lines 1100–1151 A loop inside a loop

In a **nested loop**, the inner loop runs **completely** for every single round of the outer loop. With 4 people and 3 skills, the inner `print` runs 4 × 3 = 12 times.

```
for name in peoples:            # outer: 4 rounds
  print(f"{name} Skills Is: ")
  for skill in skills:          # inner: 3 rounds, for EACH name
    print(f"- {skill}")
```

Nested loops over a nested dictionary (lines 1119–1151)

Here the outer dict's values are dicts themselves. The outer loop walks through names; the inner loop walks through the keys of *that person's* dict:

```
for name in peoples:
  for skill in peoples[name]:                        # keys of the inner dict
    print(f"{skill.upper()} => {peoples[name][skill]}")
```

```
OUTPUT
Skills and Progress For Osama Is:
HTML => 70%
CSS => 80%
JS => 70%
...
```

Read `peoples[name][skill]` from left to right: "from `peoples`, take this person's dict, then from it take this skill's value".

28 Loop Control: break, continue, pass

Lines 1157–1195 Jump statements

All three use the list `[1, 2, 3, 5, 7, 10, 13, 14, 15, 19]` and react when `number == 13`:

KEYWORD	WHAT HAPPENS AT 13	PRINTED
`continue`	Skips the **rest of this round** and goes to the next item.	everything **except** 13
`break`	**Exits the loop** immediately. A loop `else` would be skipped.	1, 2, 3, 5, 7, 10 then stops
`pass`	**Does nothing.** It's a placeholder for code you'll write later.	everything, **including** 13

Why does `pass` exist? Python doesn't allow an empty block. If you write `if x:` with nothing under it, you get an error. `pass` fills the hole: `def todo(): pass`.

UNDERSTANDING THE PASS EXAMPLE

Lines 1189–1195 print 13 like every other number, because `pass` doesn't skip anything. The example shows that `pass` has no effect on the flow, which is the difference from `continue`.

COMING FROM JAVASCRIPT

`break` and `continue` are identical to JS. `pass` is like an empty `{}` block.

29 Iterating Dictionaries with `items()`

Lines 1198–1242 Key–value iteration · tuple unpacking in loops

`mySkills.items()` gives `(key, value)` tuples. Combine that with tuple unpacking (section 11) and you get both at once, without looking the value up by key:

```
# way 1: loop over keys, look up value
for skill in mySkills:
  print(f"{skill} => {mySkills[skill]}")

# way 2: unpack each (key, value) pair   <- preferred
for skill_key, skill_progress in mySkills.items():
  print(f"{skill_key} => {skill_progress}")
```

Both print the same thing. Way 2 is shorter, faster and easier to read, and it's the style you'll see in real code.

Nested dict with `items()` twice (lines 1225–1242)

```
for main_key, main_value in myUltimateSkills.items():
  print(f"{main_key} Progress Is: ")
  for child_key, child_value in main_value.items():   # main_value is a dict
    print(f"- {child_key} => {child_value}")
```

```
OUTPUT
HTML Progress Is:
- Main => 80%
- Pugjs => 80%
CSS Progress Is:
- Main => 90%
- Sass => 70%
```

> **COMING FROM JAVASCRIPT**
>
> This is `for (const [key, value] of Object.entries(obj))`.

30 Defining Functions and `return`

Lines 1244–1264 `def` · return values · DRY

A **function** is a named, reusable block of code. You **define** it once with `def` and **call** it as many times as you like with parentheses. Your comment block (lines 1247–1255) is a good summary. "DRY" means **Don't Repeat Yourself**: if you write the same code twice, make it a function.

```
def function_name():
  return "Hello Python From Inside Function"

dataFromFunction = function_name()   # call it and keep the result
print(dataFromFunction)
```

- `def` + name + `()` + `:`, then an indented body.
- `return` sends a value back to whoever called the function and **ends** the function immediately.
- A function without `return` (or with a bare `return`) gives back `None`.
- A function must be defined **before** the line that calls it runs.

PRINT VS RETURN

`print` shows something on the screen and that's it. `return` hands a value back so the rest of your program can use it. Most of the functions in your file print; backend code mostly returns.

COMING FROM JAVASCRIPT

`def name():` is `function name() { }`. Same `return` behaviour; `None` plays the role of `undefined`.

31 Parameters and Arguments

Lines 1267–1315 Passing data into functions

The comments on lines 1277–1281 nail the vocabulary:

- A **parameter** is the variable name in the definition: `def say_hello(n):` → `n`.
- An **argument** is the actual value you pass when calling: `say_hello("Ahmed")` → `"Ahmed"`.

Arguments are matched to parameters by **position**: first to first, second to second. Python checks the count: calling `addition(1)` when it needs two raises a `TypeError`.

You can also pass by **name** (keyword arguments), which makes order irrelevant: `full_name(last="elsayed", first="osama", middle="mohamed")`.

Validating input (lines 1299–1309)

```python
def addition(n1, n2):
  if type(n1) != int or type(n2) != int:
    print("Only Integers Allowed")
  else:
    print(n1 + n2)
```

This guards the function against bad input. The more idiomatic check is `isinstance(n1, int)`, which you'll see in most Python code.

> **REDEFINING A FUNCTION**
> `addition` is defined twice (lines 1291 and 1299). Python doesn't complain: the second `def` simply replaces the first from that point on. The same happens with `say_hello` (4 times) and `show_skills` (3 times) later in the file. It's fine for practice, but in a real program it's a bug waiting to happen, so give each function its own name.

Format specs on method results (line 1313)

```python
print(f"Hello {first.strip().capitalize()} {middle.upper():.1s} {last.capitalize()}")
```

Inside the braces you can call methods and then apply a format spec: `middle.upper():.1s` means "uppercase it, then keep 1 character". Output: `Hello Osama M Elsayed`.

32 Variadic Positional Arguments: *args

Lines 1317–1345 Packing and unpacking with `*`

The star `*` has two opposite jobs, depending on where you write it.

Unpacking (when calling)

`*myList` spreads a list into separate arguments:

```
print(myList)     # [1, 2, 3, 5]    one argument: the list
print(*myList)    # 1 2 3 5         same as print(1, 2, 3, 5)
```

Packing (when defining)

`*peoples` in a definition collects **any number** of positional arguments into a single **tuple**:

```
def say_hello(*peoples):
  for name in peoples:          # peoples = ("Osama", "Ahmed", "Sayed", "Mahmoud")
    print(f"Hello {name}")

say_hello("Osama", "Ahmed", "Sayed", "Mahmoud")
```

You can mix normal parameters with it. Normal ones are filled first; `*skills` takes whatever's left:

```
def show_details(name, *skills): ...
show_details("Osama", "Html", "CSS", "JS")   # name="Osama", skills=("Html","CSS","JS")
```

The name after the star can be anything. `*args` is just the common convention.

> **COMING FROM JAVASCRIPT**
> Unpacking is the spread operator `fn(...arr)`. Packing is rest parameters `function f(name, ...skills)`. The only differences: Python uses one star instead of three dots, and the packed value is a tuple instead of an array.

33 Default Parameter Values

Lines 1348–1360 Optional parameters

Give a parameter a value in the definition and it becomes **optional**. If the caller leaves it out, the default is used.

```
def say_hello(name="Unknown", age="Unknown", country="Unknown"):
  print(f"Hello {name} Your Age is {age} and Your Country Is {country}")

say_hello("Sameh", 38)   # country falls back to its default
say_hello()              # everything falls back
```

```
OUTPUT
Hello Sameh Your Age is 38 and Your Country Is Unknown
Hello Unknown Your Age is Unknown and Your Country Is Unknown
```

Rule: parameters with defaults must come **after** those without. `def f(a=1, b):` is a `SyntaxError`.

A FAMOUS TRAP

Never use a mutable value like `[]` or `{}` as a default (`def add(item, bucket=[])`). The same list is reused across every call and keeps growing. Use `bucket=None` , then `if bucket is None: bucket = []` inside.

COMING FROM JAVASCRIPT

Same syntax as JS default parameters: `function sayHello(name = "Unknown")` .

34 Variadic Keyword Arguments: **kwargs

Lines 1363–1393 Packing and unpacking with `**`

Two stars do for **named** arguments what one star does for positional ones. Lines 1367–1375 first remind you that `*skills` is a `tuple`; lines 1385–1393 show that `**skills` is a `dict`.

```
def show_skills(**skills):
  print(type(skills))                  # <class 'dict'>
  for skill, value in skills.items():
    print(f"{skill} => {value}")

show_skills(**mySkills)                # unpack a dict into named arguments
show_skills(Html="80%", Css="70%")     # same thing, written by hand
```

- **In a definition**, `**skills` **packs** any extra `name=value` arguments into a dict.
- **In a call**, `**mySkills` **unpacks** a dict so each key becomes an argument name. The keys must be strings that are valid names.

`**kwargs` ("keyword arguments") is the conventional name. You'll see it everywhere in frameworks like Django and FastAPI, where functions accept flexible options.

COMING FROM JAVASCRIPT

JS has no named arguments, so the usual pattern is passing an options object: `fn({ html: "80%" })` and destructuring it. `**` unpacking a dict is a bit like `{ ...obj }` spread.

35 Combining name, *args and **kwargs

Lines 1396–1422 Parameter order

```
def show_skills(name, *skills, **skillsWithProgres):
  ...

show_skills("Osama", *myTuple, **mySkills)
```

On the call line, Python expands everything first: `"Osama"` goes to `name`, the three items of `*myTuple` fill `skills`, and the three pairs of `**mySkills` fill `skillsWithProgres`.

```
OUTPUT
Hello Osama
Skills Without Progress Is:
- Html
- CSS
- JS
Skills With Progress Is:
- Go => 80%
- Python => 50%
- MySQL => 80%
```

The order rule for parameters in a definition is always:

```
def f(normal, default=1, *args, **kwargs): ...
```

The `\n` in line 1410 is the **newline** escape sequence: it breaks the line inside a single string, which is why "Skills Without Progress Is:" is on its own line.

36 Variable Scope and `global`

Lines 1426–1449 Local vs global scope

Scope is where a variable name can be seen. A variable created at the top level of the file is **global**. A variable assigned inside a function is **local** to that function: it's created when the function runs and disappears when it returns, and it doesn't touch a global variable of the same name.

```
x = 1                       # global

def one():
  global x                  # "x here means the GLOBAL x"
  x = 2                     # so this changes the global

def two():
  x = 10                    # a NEW local x; the global is untouched
  print(...)
```

Follow the calls in order:

```
OUTPUT
Print Variable From Function Scope 2                  # one(): set global x to 2
Print Variable From Global Scope 2                    # global x is now 2
Print Variable From Function Scope 10                 # two(): its own local x
Print Variable From Global Scope After One Function Is Called 2
```

Python looks names up in the order **L**ocal → **E**nclosing function → **G**lobal → **B**uilt-in (the "LEGB rule"). Reading a global inside a function works without any keyword; only **assigning** to it needs `global`.

GOOD PRACTICE

Use `global` sparingly. Functions are easier to test and reuse when they take inputs as parameters and give results back with `return`, rather than changing global variables.

COMING FROM JAVASCRIPT

Python has no block scope: a variable created inside an `if` or a `for` is still visible after it, in the same function. Only functions (and modules, classes) create a new scope. And in JS, assigning to an outer variable just works; in Python you need `global` (or `nonlocal` for an enclosing function).

37 Recursion

Lines 1452–1481 A function that calls itself

Recursion means solving a problem by having a function call itself on a **smaller** version of the same problem. Every recursive function needs two parts:

1. A **base case**: a situation simple enough to answer directly, which stops the recursion.
2. A **recursive case**: do a bit of work, then call itself on a smaller input that moves toward the base case.

```
def cleanWord(word):
  if len(word) == 1:                    # base case: one letter, nothing to clean
    return word
  if word[0] == word[1]:                # first two letters the same?
    return cleanWord(word[1:])          #   drop the first, recurse on the rest
  return word[0] + cleanWord(word[1:])  # keep the first, recurse on the rest
```

The function removes repeated neighbouring letters. `word[1:]` is a slice: the word without its first letter, so the input gets one letter shorter each call and must eventually hit the base case.

Tracing `"WWWoooorrrldd"`

CALL WITH	FIRST TWO EQUAL?	KEEPS
WWWoooorrrldd	yes (W W)	–
WWoooorrrldd	yes (W W)	–
Woooorrrldd	no	W
ooorrrldd → oorrrldd → orrrldd	yes, yes, yes	–
orrrldd	no	o
rrrldd → rrldd	yes, yes	–
rldd	no	r
ldd	no	l
dd	yes	–
d	base case → returns `"d"`	

Then the calls **return** back up, each one gluing its kept letter in front: `"W" + "o" + "r" + "l" + "d"` → `World`. Your comment `# Stash [ World ]` is pointing at exactly this: the kept letters wait in a stack until the base case returns. The `print` lines inside the function show the order of calls, which is why the output has 24 "Print..." lines before `World`.

LIMITS

Python stops recursion at about 1000 nested calls (`RecursionError`). For simple repetition a loop is usually better; recursion shines for naturally nested data, like folders inside folders or JSON inside JSON.

38 Lambda (Anonymous) Functions

Lines 1483–1505 Single-expression functions

A **lambda** is a small function written in one expression, without `def` and without a name. It automatically returns the value of its expression (no `return` keyword).

```
def say_hello(name, age) : return f"Hello {name} your Age Is: {age}"
hello = lambda name, age : f"Hello {name} your Age Is: {age}"
#       lambda PARAMETERS : EXPRESSION
```

Both calls print the same thing. The difference shows in `__name__`:

```
OUTPUT
say_hello
<lambda>
<class 'function'>
```

A `def` function knows its name. A lambda is anonymous: even after you store it in `hello`, its own name is `<lambda>`. Its type is still an ordinary `function`.

The rules in your comments (lines 1487–1492) are correct. The most important one: a lambda can only hold **one expression**. No loops, no multiple statements, no assignments.

Where you'll actually use it

Lambdas are mostly passed directly into other functions that need a small "rule":

```
names = ["sara", "Ali", "mohammed"]
sorted(names, key=lambda n: len(n))          # sort by length
list(map(lambda x: x * 2, [1, 2, 3]))        # [2, 4, 6]
list(filter(lambda x: x > 1, [1, 2, 3]))     # [2, 3]
```

STYLE NOTE

Assigning a lambda to a name, like line 1497, works but PEP 8 recommends a `def` in that case. Use lambdas inline, as above.

COMING FROM JAVASCRIPT

A lambda is like a one-line arrow function: `(name, age) => `Hello ${name}``. Unlike arrow functions, a lambda can never have a `{ }` body with several statements.

39 Practice: Three Ways to Call `range()`

Lines 1510–1523 Index-based loops

Your last block shows that these three loops are identical. Each prints 0 to 9:

```
for i in range(0, 10, 1):   # start, stop, step all written out
for i in range(10):         # start defaults to 0, step to 1
for i in range(0, 10):      # step defaults to 1
```

Then it uses `range(len(arr))` to loop over a list **by index**, the way a classic JS `for (let i = 0; i < arr.length; i++)` loop does:

```
arr = [1, 2, "moayad"]
for i in range(0, len(arr), 1):
    print(arr[i])
```

That works, but in Python you only need the index when you really use it. Two more Pythonic versions:

```
for item in arr:                # just the items (most common)
    print(item)

for i, item in enumerate(arr):  # index AND item together
    print(i, item)              # 0 1 / 1 2 / 2 moayad
```

`enumerate()` is the tool to reach for when you think "I need the index too".

★ Cheat Sheet

Whole file One page to glance at

Collections

	SYNTAX	ORDERED	CHANGEABLE	DUPLICATES
list	`[1, 2]`	yes	yes	yes
tuple	`(1, 2)`	yes	no	yes
dict	`{"k": 1}`	yes*	yes	keys no
set	`{1, 2}`	no	yes	no
str	`"ab"`	yes	no	yes

* insertion order, since Python 3.7

Slicing `seq[start:stop:step]`

`[1:4]` items 1–3 · `[:3]` first 3 · `[-1]` last · `[::-1]` reversed · stop is excluded

Formatting

`f"{x}"` · `{x:.2f}` 2 decimals · `{x:,d}` 1,000 · `{s:.3s}` first 3 chars · `"{} {}".format(a, b)` · `"%s %d" % (a, b)`

Falsy values

`0` `0.0` `""` `[]` `()` `{}` `set()` `None` `False`

Operators

Arithmetic `+ - * / // % **`
Compare `== != > < >= <=`
Logical `and or not`
Membership `in`, `not in`
Assign `= += -= *= /= //= %= **=`

Control flow

```
if a: ...
elif b: ...
else: ...
x = "yes" if cond else "no"
for item in items: ...
for k, v in d.items(): ...
for i in range(10): ...
while cond: ...
break / continue / pass
```

Functions

```
def f(a, b=2, *args, **kwargs):
    return a + b
f(1)  f(1, 3)  f(*lst)  f(**dct)
g = lambda x: x * 2
```

JavaScript → Python

`let x = 1`	`x = 1`	`arr.push(x)`	`lst.append(x)`
`` `Hi ${n}` ``	`f"Hi {n}"`	`arr.includes(x)`	`x in lst`
`&& || !`	`and or not`	`arr.length`	`len(lst)`
`c ? a : b`	`a if c else b`	`for (const x of a)`	`for x in a:`
`null / undefined`	`None`	`Object.entries(o)`	`d.items()`
`true / false`	`True / False`	`(...args) =>`	`lambda *args:`
`x++`	`x += 1`	`Math.floor(a/b)`	`a // b`

✔ Suggested Fixes in Your File

Summary Everything flagged in the sections above, in line order

LINE	WHAT'S HAPPENING	SUGGESTION
71	Comment says `%d => Number`.	`%d` is for integers; `%f` is for decimals.
161	Missing `f` prefix, so `{myName}` prints literally.	Add `f`, or a comment saying it's a deliberate "wrong" example.
455	Comment says dicts aren't ordered.	Since Python 3.7 they keep insertion order.
777	A set appears without introduction; its print order can vary.	See section 18; don't rely on set order.
825–827	Prompts have no space at the end.	`input("First name? ")`
846–1242	Mix of 2-space and 4-space indentation.	Use 4 spaces everywhere (PEP 8).
872	`isStudent = "Yes"` as a string.	Use a real boolean: `isStudent = True`.
877, 890	Repeated `== … or == …`.	`if uCountry in ["Egypt", "KSA", "Qatar"]:`
901–908	Heading says "ternary" over one-line ifs.	The real ternary is line 923.
1291/1299, 1283/1328/1352/1495, 1367/1385/1408	Same function name defined several times; each new `def` replaces the old one.	Give each version its own name.
1301	`type(n1) != int`	`not isinstance(n1, int)`
1497	Lambda assigned to a name.	Fine for learning; in real code prefer `def` here.
1522	`range(len(arr))` to read items.	`for item in arr:` or `enumerate(arr)`.
Throughout	`camelCase` names like `myName`.	Python convention is `snake_case`: `my_name`.

Every output in this guide comes from running weeekone.py with Python 3.13, using the sample input "moayad / ahmad / attalah".